

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №9

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Поиск расстояний в графе"

Выполнил:

Студент группы 24ВВВ3

Макунькин А.М.

Мелюшев М.М.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение поиска расстояний в графе.

Теоретическая часть

Поиск расстояний – довольно распространенная задача анализа графов.

Для поиска расстояний можно использовать процедуры обхода графа. Для этого при каждом переходе в новую вершину необходимо запоминать, сколько шагов до нее мы сделали. При этом вектор, который хранил информацию о посещении вершин становится вектором расстояний. Довольно просто модернизировать для поиска расстояний в графе алгоритм обхода в ширину, т.к. этот алгоритм проходит вершины по уровням удаленности, то для не ориентированного графа для вершин каждого следующего уровня глубины расстояние от исходной вершины увеличивается на 1. Удалённость в данном случае понимается как количество ребер, по которым необходимо прейти до достижения вершины.

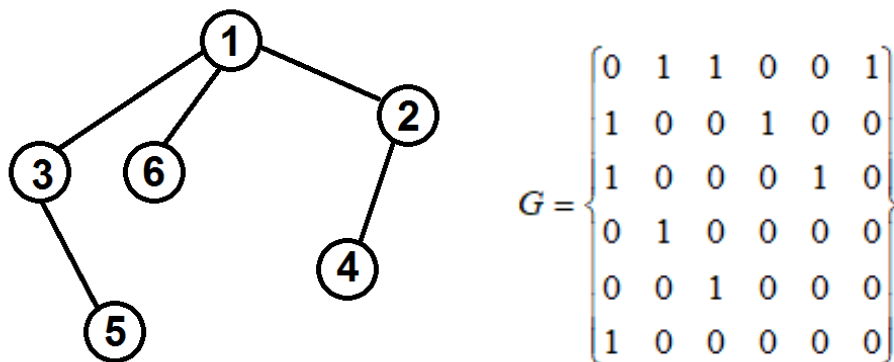


Рисунок 1 – Граф

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

Вход: G – матрица смежности графа, v – исходная вершина.

Выход: DIST – вектор расстояний до всех вершин от исходной.

Алгоритм ПОШ

1.1. для всех i положим DIST [i] = -1 пометим как "не посещенную";

1.2. **ВЫПОЛНЯТЬ** BFS (v).

1.3 для всех i вывести DIST [i] на экран;

Алгоритм BFS (v):

- 2.1. Создать пустую очередь $Q = \{\}$;
- 2.2. Поместить v в очередь $Q.push(v)$;
- 2.3. Обновить вектор расстояний $DIST[x] = 0$;
- 2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**
- 2.5. $v = Q.front()$ установить текущую вершину;
- 2.6. Удалить первый элемент из очереди $Q.pop()$;
- 2.7. вывести на экран v ;
- 2.8. **ДЛЯ** $i = 1$ **ДО** $size_G$ **ВЫПОЛНЯТЬ**
- 2.9. **ЕСЛИ** $G(v,i) == 1$ **И** $DIST[i] == -1$
- 2.10. **ТО**
- 2.11. Поместить i в очередь $Q.push(i)$;
- 2.12. Обновить вектор расстояний $DIST[i] = DIST[v] + 1$;

Лабораторное задание:

- а. Сгенерировать (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Вывести матрицу на экран.
- б. Для сгенерированного графа осуществить процедуру поиска расстояний. При реализации алгоритма в качестве очереди использовать класс `queue` из стандартной библиотеки $C++$.

Результат программы

```
Консоль отладки Microsoft Vi X + v
Input number of vertission : 6
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 1 0 1
0 0 1 0 0 0
0 0 0 0 0 1
1 1 1 0 1 0
Input start vert : 1
Path: 1 5 0 2 4 3

Distance from 1 to:
0 : 3
1 : 1
2 : 3
3 : 4
4 : 3
5 : 2
```

Рисунок 1 – Поиск расстояний графа

Листинг

Файл 1-Lab3L.c

```
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <stdio.h>
#include <conio.h>
#include <cstdlib>
#include <ctime>
#include <queue>

using namespace std;

void BFSD(int** G, int numG, int* dist, int s) {
    queue<int> q;
    int v;
    dist[s] = 1;
    q.push(s);

    while (!q.empty()) {

        v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && dist[i] == -1) {
                q.push(i);
                dist[i] = dist[v] + 1;
            }
        }
    }

    printf("\n\nDistance from %d to: ", s);
    for (int i = 0; i < numG; i++) {
        printf("\n%3d : %3d", i, dist[i]);
    }
}

int main() {

    int** G;
    int* distance;
    int numG, current;

    srand(time(NULL));

    printf("Input number of vertission : ");
    scanf("%d", &numG);

    distance = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for (int i = 0; i < numG; i++) {
        distance[i] = -1;
        for (int j = i; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
```

```

        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }

    printf("Input start vert : ");
    scanf("%d", &current);

    printf("Path: ");

    BFSD(G, numG, distance, current);
    printf("\n\n");

    for (int i = 0; i < numG; i++)
        free(G[i]);
    free(G);
    free(distance);

    return 0;
}

```

Вывод

В ходе выполнения лабораторной работы была разработана программа для выполнения заданий Лабораторной работы №9. В процессе выполнения работы были использованы знания о поиске расстояний в графе.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/LareklAREK/LiOAiIZ-2OURSE-.git>

<https://github.com/Motorrr/LiOAvIZ.git>